

山东大学__计算机科学与技术__学院
__大数据分析实践__课程实验报告

学 号 : 202300130059	姓名: 林子洋	班级: 23 数据
实验题目: BERT 实验		
实验学时: 2	实验日期: 2025.11.1	
实验目的: 掌握基于 Hugging Face 生态使用预训练 BERT 模型完成文本同义预测任务的流程。 实现 MRPC 数据集的加载、预处理、模型微调与推理验证, 验证 BERT 在同义句对分类任务中的效果。 理解预训练模型在下游任务中的适配方法, 分析 BERT 在文本语义匹配任务中的性能。		
硬件环境: 计算机一台		
软件环境: Linux, AutoDL 算力云		

实验步骤与内容：

环境准备：安装 transformers、datasets 等依赖库，配置 Python 运行环境。

数据处理：加载 GLUE-MRPC 数据集（5800 个句子对，二元标签区分同义 / 不同义），用 BERT 分词器对句子对进行截断、填充，统一为 128 长度的输入格式。

模型初始化：加载 bert-base-uncased 预训练模型，适配为 2 分类任务（同义 / 不同义）。

模型微调：设置训练参数（学习率 $2e-5$ 、batch_size=16、训练 3 轮），以 Accuracy 和 F1 为评价指标，在 MRPC 训练集上微调模型。

推理验证：构建同义预测 Pipeline，输入测试句子对，输出分类结果及置信度。

实验代码如下：

```
# 安装依赖
```

```
# !pip install transformers datasets evaluate
```

```
from transformers import (  
    AutoTokenizer,  
    AutoModelForSequenceClassification,  
    TrainingArguments,
```

```

        Trainer,
        pipeline
    )
from datasets import load_dataset
import evaluate
import numpy as np

# 1. 加载 MRPC 数据集
dataset = load_dataset("glue", "mrpc") # 加载 GLUE 的 MRPC
子集
print("MRPC 数据集结构：", dataset)

# 2. 加载 BERT 模型与分词器
model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model =
AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=2) # 2 分类（同义/不同义）

```

3. 数据预处理（对句子对进行分词）

```
def preprocess_function(examples):  
    return tokenizer(  
        examples["sentence1"],  
        examples["sentence2"],  
        truncation=True, # 超过最大长度则截断  
        padding="max_length", # 填充到最大长度  
        max_length=128  
    )
```

```
tokenized_dataset = dataset.map(preprocess_function,  
    batched=True)
```

4. 定义评价指标（MRPC 用 Accuracy 和 F1）

```
metric = evaluate.load("glue", "mrpc")
```

```
def compute_metrics(eval_pred):  
    logits, labels = eval_pred  
    predictions = np.argmax(logits, axis=-1)  
    return metric.compute(predictions=predictions,  
        references=labels)
```

5. 训练模型

```
training_args = TrainingArguments(  
    output_dir="./mrpc-bert",  
    learning_rate=2e-5,  
    per_device_train_batch_size=16,  
    per_device_eval_batch_size=16,  
    num_train_epochs=3,  
    weight_decay=0.01,  
    evaluation_strategy="epoch", # 每个 epoch 评估一次  
    save_strategy="epoch",  
    load_best_model_at_end=True  
)  
  
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=tokenized_dataset["train"],  
    eval_dataset=tokenized_dataset["validation"],  
    compute_metrics=compute_metrics  
)
```

```
trainer.train() # 开始微调

# 6. 同义预测 Pipeline（推理）
paraphrase_classifier = pipeline(
    "text-classification",
    model=model,
    tokenizer=tokenizer,
    device=0 if trainer.args.device.type == "cuda" else -1 # 自动选择设备
)

# 测试示例
test_example = {
    "sentence1": "The cat sat on the mat.",
    "sentence2": "The feline rested on the rug."
}
result = paraphrase_classifier((test_example["sentence1"],
test_example["sentence2"]))
print("\n 预测结果：", result)
print("是否同义：", "是" if result[0]["label"] == "LABEL_1" else "否")
```

结论分析与体会：

结论分析：

模型性能：微调后的 BERT 在 MRPC 验证集上可达到 85% 以上的 Accuracy 和 F1 值，能有效区分同义句对。

方法有效性：预训练模型通过适配下游任务（分类头替换）、数据预处理（句子对拼接），可快速迁移到语义匹配任务。

局限性：对训练数据量有一定依赖，短文本语义差异较小的句子对可能出现误判。

体会：

1：Hugging Face 生态大幅简化了预训练模型的下游任务适配流程，降低了自然语言处理任务的实现门槛。

2：BERT 的预训练语义表示能力在文本匹配任务中表现突出，验证了预训练 - 微调范式的有效性。

3：实际应用中需注意数据预处理的合理性（如句子对的拼接方式），同时可通过调整模型参数（如更大的预训练模型、更

多训练轮次) 进一步提升性能。